

MODULE 1

Foundations

The Renaissance Developer & Your AI Toolkit

Renaissance Developer Academy — Detailed Lesson Plan

Week 1 of 10 | 5 Days | Full-Time Intensive
Badge Earned: Renaissance Foundations

v1.1 — Revised March 2026

Module Overview

Module 1 establishes who the Renaissance Developer is, why this archetype matters in the AI era, and how the program will transform mid-level engineers into polymath builders. The week opens with a deep conceptual session on the Renaissance Developer framework — the five core mindsets, the competency layers, and the AI multiplier model — then moves into hands-on mastery of AI development tools. By Friday, every student will have shipped their first AI-augmented application to production.

Important structural note: GitHub skills and tool installation are treated as an **add-on bootcamp** (see Appendix C). Students who are already proficient with Git, GitHub Actions, and have their tools installed may skip the add-on. Students who need foundational support complete it before or alongside Week 1. This keeps the core curriculum focused on the concepts and skills that are unique to this program.

Learning Objectives

1. Understand the Renaissance Developer archetype: the five mindsets, four competency layers, and how they map to the program curriculum
2. Internalize the AI multiplier model: AI Generation → Human Verification → Business Value
3. Adopt the Creative Director mindset: architect, verify, and edit AI output rather than writing every line
4. Set up and navigate all five AI development tools (Claude Code CLI, Kiro, Antigravity, OpenClaw/Nanobot, NotebookLM)
5. Ship a first AI-augmented application from idea to deployed production with automated CI/CD
6. Build a personal NotebookLM notebook loaded with all course materials for ongoing AI-assisted study

Pre-Work

Due before Day 1. Estimated time: 2–4 hours (more if completing the GitHub Add-On).

1. **GitHub & Tool Add-On (if needed):** Complete the GitHub Skills Bootcamp (Appendix C) covering Git fundamentals, branching, PRs, GitHub Actions, and GitHub Projects. Install all development tools per the setup guide. Students already comfortable with Git + GitHub Actions may self-certify and skip.
2. **Self-assessment survey:** Complete the pre-program skills self-assessment covering Git proficiency, languages known, AI tool familiarity, business exposure, and system design experience.
3. **Pre-reading:** Read the Renaissance Developer role summary (2 pages, provided). Come prepared to discuss: what excites you and what intimidates you about this role definition?

4. **NotebookLM access:** Sign into notebooklm.google.com. Create your first notebook and upload the role summary PDF to verify it works.

Week-at-a-Glance

Day	Theme	Primary Tools	Key Output
Day 1	The Renaissance Developer: Concept & Vision	Slides, Discussion, NotebookLM	Personal development map, cohort norms
Day 2	AI Tool Mastery: Your New Toolkit	Claude Code, Kiro, Antigravity	All tools configured, Rosetta Stone comparison
Day 3	Agent Architecture & OpenClaw Ecosystem	Nanobot, OpenClaw docs, Claude Code	Working Nanobot agent, architecture analysis
Day 4	The One-Day Ship Challenge	All AI tools, GitHub Actions	Full app deployed with CI/CD
Day 5	NotebookLM, Reflection & Calibration	NotebookLM, all tools	Study notebook, blog post, self-assessment

Day 1: The Renaissance Developer — Concept & Vision

Theme: Who is the Renaissance Developer, why this archetype matters now, and where you fit on the journey

Goal: Every student understands the five core mindsets, the four competency layers, and the AI multiplier model. Each student creates a personal development map identifying their starting strengths and growth edges.

Schedule

09:00	Welcome & Program Orientation	45 min
	- Program vision: not just another bootcamp — we're training a new archetype of engineer 10-week roadmap overview — the arc from foundations to capstone - Cohort introductions: name, background, what you hope to build by Week 10 - Logistics: daily rhythm, Slack, office hours, how Fridays work (Impossible Stuff Day) - The “startup CTO” metaphor: you will bridge product building and business problems	
09:45	The Renaissance Developer Archetype (Concept Session)	90 min
	- PART 1 — Core DNA: The Five Mindsets (30 min) — Curiosity (保持好奇): shape a culture of continuous learning; discover problems, not just solve them — Outcome (结果导向): define accountability standards focused on business impact, not just code shipped — Polymath (博学思维): cross-disciplinary learning; think like a business insider; translate specialist knowledge into accessible explanations — Precision (精准沟通): spec-driven development; bridge C-level executives to frontline workers with clarity — Systems (系统思考): create order from complexity; connect technical systems with business processes through chaos engineering - Exercise: Self-rate on the radar chart (1–5 per mindset). Where are you strong? Where's the gap? Share with a partner. - PART 2 — The Four Competency Layers (30 min) — Layer 1: Technical Foundation — Cloud, Data Modeling, DevOps/CI/CD, Lean Thinking, Tech Debt (the non-negotiable engineering literacy) — Layer 2: Architect — Architecture & Design, Code Quality, SRE, Service Management (building stability and scale) — Layer 3: Enabler — Pattern Generalization, Developer Experience, Knowledge Management, Technical Writing (multiplying from individual to team impact)	

	— Layer 4: Strategist & Leader — Business Immersion, Data Integration, Problem Discovery, Rapid Prototyping, Multi-Audience Communication, Stakeholder Management (driving business and data complexity, influencing beyond code) - Discussion: how does each layer map to the 10-week curriculum? Which layers feel familiar vs. foreign? - PART 3 — The AI Multiplier Model (30 min) — The three-stage flow: AI Generation → Human Verification (Quality Gate) → Business Value — Augmented Development: design AI workflows, evaluate tools, balance speed with verification rigor — Literacy & Governance: guide AI adoption, assess risks, design human-in-the-loop workflows — Quality Gate: verify and review AI-generated code to ensure it meets high standards — The Creative Director metaphor: you are the architect, AI is your construction crew — Key insight: AI shifts the core competency from writing code to architecting, validating, and integrating
--	--

11:15	Break	15 min
	Stretch, coffee, absorb	

11:30	The Developer Manifesto & Responsibility Matrix	45 min
	- The Developer Manifesto: Polymath at the center — Curiosity, Outcome, Precision, Systems as pillars - The Responsibility Matrix: six domains the Renaissance Developer owns simultaneously: — Delivery: critical technical initiatives, engineering delivery practices — People: developing technical leaders, shaping talent strategy — Business: technology-driven business strategy, executive-level representation — AI: workflow design, tool evaluation, training engineers — Reliability: SLOs/SLIs, resilient systems, reliability culture — Process: efficiency, cross-functional coordination, retrospectives - The Core Value: creating clarity from chaos (在混沌中建立清晰) — Challenge: organizational complexity, ambiguous business problems — Action: strategic direction, architectural guidance, outcome accountability — Result: clarity, defined practices, external industry visibility - Group discussion: which responsibility domain feels most natural? Which feels most foreign?	

12:15	Lunch	60 min
	Encourage informal discussions about the morning's frameworks	

13:15	Personal Development Map Workshop	90 min
	• EXERCISE 1: Create Your Renaissance Developer Personal Map • Using the frameworks from the morning, each student creates a structured self-assessment: — Part A: Rate yourself on each of the five mindsets (1–5) with evidence from your career — Part B: For each of the four competency layers, list your existing skills and identify gaps — Part C: Plot yourself on the Strategist Venn diagram — are you stronger on Business Strategy or Technical Execution? Where's the overlap? — Part D: Write your “10-week intention”: What specific transformation do you want by graduation? • Pair share: exchange maps with a partner and give each other honest calibration feedback • Instructor circulates for 1:1 conversations — this is the first relationship-building moment • EXERCISE 2: Cohort Norms — Facilitated discussion: What agreements should we make about how we work together? — Example norms: “Review PRs within 4 hours”, “Ask for help before 30 min stuck”, “Demos over decks” — Document norms in a shared GitHub repo (the cohort “operating agreement”)	
14:45	Break	15 min
	• Stretch	
15:00	Forward-Deployed Engineering: Case Studies & Role Models	60 min
	• The Palantir FDE model: engineers embedded directly with business units, operating like startup CTOs — Small teams (5–10), end-to-end ownership, simultaneous product engineering + customer engagement — Production access from Day 1, platform-first philosophy, field-to-product feedback loop • How other companies adopted this: OpenAI, Ramp, Salesforce, Gecko Robotics • Shopify’s “spiky generalist” model: default to the most important work, not prescribed roles • Stripe’s generalist engineers: work fullstack, spend time directly with customers • Discussion: what would it look like to apply this model in YOUR previous company? • Key takeaway: the Renaissance Developer is not theoretical — companies are hiring for this right now	
16:00	Tool Landscape Overview & GitHub Checkpoint	45 min

	• Quick overview of the five AI tools we'll use and when each shines: • — Claude Code CLI: terminal-native, Unix composable, for daily coding and CI/CD integration • — Kiro: spec-driven development, requirements → design → tasks → code pipeline • — Antigravity: multi-agent orchestration, Manager View for parallel development • — OpenClaw ecosystem: autonomous agents, our full-stack case study through the program • — NotebookLM: AI-powered study companion, audio overviews, flashcards, quizzes • GitHub Checkpoint: quick pulse — who completed the add-on bootcamp? Who needs help tonight? • — Verify: everyone has a GitHub repo, can push code, and has tools installed • — Pair struggling students with helpers for an evening troubleshooting session • HOMEWORK: Ensure all tools are installed and working. If not done, complete the GitHub Add-On tonight. • HOMEWORK: Upload the Renaissance Developer role summary to your NotebookLM notebook
--	---

16:45	Day 1 Close	15 min
	• Reflection: one sentence — “The thing that most surprised me about the Renaissance Developer concept is...” • Preview: tomorrow is fully hands-on with AI tools. Come ready to code.	

Instructor Notes — Day 1: This is the most important conceptual day of the entire program. Resist the temptation to rush through the frameworks to “get to the code.” Students need to internalize WHY before HOW. The radar chart self-assessment (Exercise 1) surfaces vulnerability early — that’s intentional. When people share their gaps honestly on Day 1, it sets the psychological safety for the whole cohort. The Personal Development Map becomes a reference document they’ll revisit on Day 5 of every module. Watch for students who rate themselves 5/5 on everything or 1/5 on everything — both patterns warrant a private conversation. The case studies session should feel inspiring, not intimidating. Emphasize: these companies hire for this role, and this program prepares you for it.

Day 2: AI Tool Mastery — Your New Toolkit

Theme: Hands-on setup and first experience with all AI development tools

Goal: Every student has all tools configured and connected to GitHub, has produced AI-assisted code with three different tools, and can articulate when to use which tool.

Schedule

09:00	The AI-Augmented Engineer (Concept Session)	45 min
	- Connecting yesterday's AI Multiplier Model to today's practice: — AI Generation: using Claude Code, Kiro, and Antigravity to generate code at speed — Human Verification: the Quality Gate — why reviewing AI code is harder than writing it yourself — Business Value: ensuring AI-generated solutions actually serve users and drive outcomes - The trust spectrum: blind acceptance → line-by-line review → strategic verification → architectural oversight - Industry context: 91% of engineering teams use AI coding tools; staff+ engineers save ~4.4 hrs/week - The 4x code clone problem: AI patterns are copied without customization — your job is to prevent this	
09:45	Claude Code CLI: Deep Dive & First Session	75 min
	- Verify installation: <code>claude --version</code>, <code>claude config show</code> - EXERCISE 3: Your first Claude Code session — navigate to your portfolio repo and try: — “Add a utility function that validates email addresses with comprehensive tests” — Review every line: correctness, edge cases, test quality, security implications — Practice: accept, reject, or modify each proposed change - Claude Code as a Unix tool: piping (<code>echo “fix linting errors” claude</code>), chaining with shell commands - Cost awareness: understanding token usage, estimating costs, setting spending limits - EXERCISE 4: Use Claude Code to generate a GitHub Actions workflow, then manually review and improve it - Key question to internalize: “What did the AI get right? What was subtly wrong? What would you never have caught without looking?”	
11:00	Break	15 min
	Questions, troubleshooting	

11:15	Kiro: Spec-Driven Development	60 min
	• Connect Kiro to your GitHub repository • The spec-driven workflow: natural language → requirements.md (EARS notation) → design.md → tasks.md → code • EXERCISE 5: Give Kiro a feature request: • — “Build a CLI tool that converts CSV files to JSON with filtering and validation” • — Review the generated specs: are requirements complete? Is the design sound? • — Let Kiro generate code, then verify against specs • Agent Hooks demo: auto-run tests on save, auto-update docs on code changes • Key insight: Kiro enforces “think before you code” — the spec IS the architecture document	
12:15	Lunch	60 min
	• Optional: tool troubleshooting office hours	
13:15	Antigravity: Multi-Agent Orchestration	60 min
	• Install and connect Antigravity to GitHub • Three development modes: Agent-driven, Review-driven (start here), Agent-assisted • EXERCISE 6: In Review-driven mode: • — “Add a REST API endpoint with input validation and error handling” • — Review each step the agent proposes before approving • — Switch to Agent-assisted mode and compare the experience • Manager View preview: orchestrating 5+ agents in parallel (used heavily in Weeks 3 and 6) • Artifacts: the audit trail Antigravity creates (task lists, plans, screenshots, browser recordings)	
14:15	The Rosetta Stone Exercise	75 min
	• EXERCISE 7 (The Rosetta Stone): Build the same feature with three different tools: • — Task: “Add a /health endpoint that returns system uptime, version, and dependency status” • — Build it with Claude Code CLI (terminal-first) • — Build it with Kiro (spec-driven) • — Build it with Antigravity (agent-assisted) • — Commit each version on a separate branch for comparison • Analysis (write up in your repo wiki): • — Which felt fastest? Which produced the best code? Which was easiest to verify? • — What surprised you about the differences? • — When would you choose each tool in a real project?	

	Class discussion: share findings. There are no wrong answers, but there ARE wrong tools for wrong situations.	
15:30	Break	15 min
	Stretch	
15:45	Prompt Engineering Foundations	45 min
	- Systematic prompt engineering — not ad hoc, but repeatable patterns: — Context setting: giving AI enough background without overwhelming — Constraint specification: language, framework, testing requirements, style guidelines — Iterative refinement: narrowing scope based on initial output — Chain-of-thought prompting: asking AI to explain its reasoning before coding - EXERCISE 8: Start building your personal prompt library — Create a prompts/ directory in your portfolio repo — Write 5 reusable prompts for common tasks: scaffolding, testing, refactoring, debugging, documentation — Commit and push — this library will grow throughout the program	
16:30	Day 2 Wrap-Up	30 min
	- Check: all five tools installed, configured, connected to GitHub - Quick share: one surprising thing about AI-generated code today - HOMEWORK: Use Claude Code to add one meaningful feature to your portfolio repo. Push as a PR. Self-review before requesting peer review. - Preview: tomorrow we dive into autonomous agent architecture with the OpenClaw ecosystem	

Instructor Notes — Day 2: This is the highest-risk day for technical issues. Have a troubleshooting guide ready for common installation failures (API keys, Docker permissions, Ollama timeouts). Designate 2–3 TAs as floating troubleshooters. The Rosetta Stone exercise (Exercise 7) is the day’s crown jewel — it builds tool selection judgment. Push students past “I liked it” to articulate WHY. If students get stuck on installation, let them pair with someone who’s working — peer debugging is itself a learning outcome.

Day 3: Agent Architecture & the OpenClaw Ecosystem

Theme: Understanding autonomous AI agents from the inside out, using the OpenClaw family as a case study

Goal: Every student has a working Nanobot agent deployed locally, understands the architecture of autonomous agents, and can articulate the security implications.

Schedule

09:00	Autonomous Agents: Architecture & Implications (Concept Session)	60 min
	• What makes an agent different from a coding assistant? • — Persistence: agents run 24/7, maintain memory across conversations • — Autonomy: agents take actions without prompt-by-prompt human oversight • — Integration: agents connect to external services (messaging, files, APIs, browsers) • The OpenClaw ecosystem as a case study — our program’s full-stack laboratory: • — Nanobot (~4K lines Python): minimal, auditable, perfect for learning agent fundamentals • — NanoClaw (~5 files): container-isolated, security-first, built on Claude Code • — ClawSwarm (multi-agent Rust/gRPC): orchestrating specialized agents via unified messaging • — OpenClaw (430K+ lines monorepo): enterprise-scale plugin architecture, 50+ integrations • Why we progress through this family: Nanobot (Weeks 2–3) → NanoClaw (4–5) → ClawSwarm (6–7) → OpenClaw (8–10) • Security reality check: 17% of ClawHub skills flagged as malicious, 135K instances exposed on the internet • — Connecting back to the AI Multiplier: Human Verification is non-negotiable for agents	

10:00	Nanobot Deep Dive: Read, Run, Understand	90 min
	• EXERCISE 9: Fork and clone the Nanobot repository from GitHub • — Read the entire codebase (it’s ~4K lines — this is achievable in one sitting) • — Map the architecture: how does a message flow from user → agent → LLM → response? • — Identify the key abstractions: tool use, memory, conversation management, model providers • EXERCISE 10: Run Nanobot locally • — Configure with a local LLM (Ollama + Qwen3 8B) or API key (Claude/OpenAI) • — Connect to a messaging platform (Telegram or Discord) • — Send messages and observe behavior • — Find where memory is stored (local Markdown files) — read what the agent “remembers”	

	— Document the architecture in a short write-up (committed to your repo) - Pair exercise: explain the Nanobot architecture to your partner without looking at code — if you can teach it, you understand it
11:30	Break 15 min
	Questions, troubleshooting agent setup
11:45	Hands-On: Extend Nanobot with a Custom Skill 75 min
	- EXERCISE 11: Add a custom skill to your Nanobot agent: — Option A: A “Project Status” skill that reads your GitHub repo’s open issues and summarizes them — Option B: A “Study Buddy” skill that quizzes you on technical concepts from a file you provide — Option C: Design your own skill based on a real problem you’ve encountered - Use Claude Code CLI to help write the skill — but verify every line (practice the AI Multiplier model) - Test the skill via your messaging platform - Push your modified Nanobot fork to GitHub with a PR describing what you built and why
13:00	Lunch 60 min
	Optional: agent hacking session for those who want to keep building
14:00	Agent Security & Ethics Workshop 60 min
	- The security reality of autonomous agents: — Case study: OpenClaw agent autonomously created a dating profile without user permission — Case study: Bitdefender found 135K+ OpenClaw instances exposed to the internet — Case study: prompt injection through malicious ClawHub skills - EXERCISE 12: Security Threat Modeling for your Nanobot agent — List all the “trust boundaries” in the system (user → messaging platform → agent → LLM → tools) — For each boundary: what could go wrong? What data could leak? What actions could be taken? — Write 3 security recommendations for anyone deploying Nanobot in production - Connection to the Renaissance Developer role: reliability and governance are core responsibilities, not afterthoughts
15:00	Break 15 min
	Stretch

15:15	CI/CD Enhancement Lab	60 min
	• EXERCISE 13: Enhance your GitHub Actions pipeline for AI-augmented development: • — Add automated testing for your Nanobot custom skill • — Add a linting step that checks code quality of AI-generated code • — (Advanced) Add a matrix build testing across Python 3.10/3.11/3.12 • — (Advanced) Add dependency caching for faster pipeline runs • For students who completed the GitHub Add-On: this is practice. For those who skipped: this is your first real pipeline. • Pair troubleshooting: debug each other's workflows before asking instructors	
16:15	Day 3 Wrap-Up	30 min
	• Share: most interesting thing you discovered reading the Nanobot codebase • HOMEWORK: Complete your Nanobot architecture write-up and push to your repo • HOMEWORK: Prepare for tomorrow's One-Day Ship Challenge: think about what you want to build • Preview: tomorrow is the big build day — zero to deployed app in one session	

Instructor Notes — Day 3: Day 3 bridges the conceptual (Day 1) and the practical (Day 2) with the program's signature case study. The Nanobot codebase is small enough to read in full — insist students actually read it, not just run it. The security workshop may feel premature, but it connects directly to the Renaissance Developer's Reliability and Governance responsibilities from Day 1. The dating profile case study gets a reaction every time — use it to drive home that autonomous agents require more verification, not less. Students who didn't complete the GitHub Add-On will get hands-on CI/CD experience in the afternoon lab.

Day 4: The One-Day Ship Challenge

Theme: Ship a complete application from idea to deployed production in one day

Goal: Every student deploys a working application with automated CI/CD. This is the program's first taste of the “deliver in days not months” mindset and the Rapid Prototyping competency.

Schedule

09:00	Ship Challenge Briefing	30 min
	- The challenge: build, test, and deploy a working web application by 4:00 PM today - Constraints: use at least 2 AI tools, CI/CD via GitHub Actions, deployed and accessible via URL - No constraints on: language, framework, cloud provider, or application type - Suggested ideas (or bring your own): — A personal dashboard pulling data from 2+ APIs — A tool solving a real problem from your career — A simple SaaS prototype (landing page + one core feature + API) — An AI-powered tool (e.g., document analyzer, code reviewer, meeting summarizer) - Success criteria: it works, it's deployed, it has tests, the CI/CD pipeline is green - Connection to the Renaissance Developer: this is the Rapid Prototyping competency in action	
09:30	Architecture & Planning Sprint	30 min
	- EXERCISE 14: Plan before coding (the Precision mindset): — 1-paragraph description: what you're building and who it serves — Architecture sketch: frontend, backend, database, external APIs — 3–5 user stories that define “done” — Tool selection: which AI tool for which part of the build? — Create GitHub Issues for each user story → add to Projects board - Instructor check: 2-minute scope review with each student — cut aggressively for ambitious scopes	
10:00	Build Sprint Block 1	150 min
	- Heads-down building. Instructors circulate for unblocking ONLY — no teaching. - Recommended sequence: backend/API → frontend → integration → tests → CI/CD → deploy - Commit every 15–20 minutes with descriptive messages 11:00 pulse check: “Who has a running backend? Who's stuck? Who needs to cut scope?”	

12:30	Working Lunch	45 min
	Keep building if you want, or recharge	
13:15	Build Sprint Block 2	105 min
	- Focus shifts to: tests (min 5), CI/CD pipeline (lint → test → build → deploy), deployment - EXERCISE 15: CI/CD must include: lint, test suite, build, deploy to any cloud 14:00 milestone: “Who has a green pipeline? Who has a deployment URL?” - Early finishers: add features, improve tests, or help a stuck peer (the People responsibility!)	
15:00	Break & Polish	15 min
	Final fixes, README updates, take a screenshot	
15:15	Lightning Demos	60 min
	- Every student: exactly 3 minutes, strictly timed: — What and why? (30 sec) — practice the Precision mindset — Live demo at deployment URL (60 sec) — Delivery in action — Show GitHub: commit history, pipeline, Projects board (60 sec) — Process transparency — One surprise from working with AI tools today (30 sec) — Curiosity reflection - Peer voting: “Most creative”, “Most technically impressive”, “Most likely to use daily”	
16:15	Retrospective	30 min
	- Structured retro: What went well? What was harder than expected? What would you change? - Key question: How did AI tools change your approach vs. building without them? - Common pattern: students underestimate CI/CD and deployment time. Why does this matter for the Delivery responsibility? - HOMEWORK: Blog post (300–500 words) in your repo wiki: “My First AI-Augmented Build — What I Learned”	

Instructor Notes — Day 4: Highest-energy day. Protect build time fiercely — no lectures. Your only job is unblocking. The most common failure is over-scoping — during planning, actively cut scope (“That’s a Week 9 capstone”). Lightning demos must be strictly 3 minutes — hard cutoff. This teaches the Precision communication the JD demands. Tie everything back to the Day 1 frameworks: the demo format maps to the six responsibility domains. Peer voting builds cohort culture.

Day 5: NotebookLM, Reflection & Calibration

Theme: Master the study companion, reflect on the week, calibrate your Personal Development Map

Goal: Every student has a fully loaded NotebookLM study environment, has revisited their Day 1 Personal Development Map, and is prepared for Module 2.

Schedule

09:00	NotebookLM Mastery Session	90 min
	- NotebookLM as your 10-week study companion: source types, 1M token context, grounded citations - EXERCISE 16: Create your Renaissance Developer Study Notebook: — Upload curriculum PDF, Module 2 pre-readings, and key reference articles — Generate Audio Overview (Deep Dive format) for Module 2 material — listen this weekend — Auto-generate flashcards for Week 1 concepts (Git, AI tools, agent architecture) — Create a self-assessment quiz on Week 1 material, take it, use “Explain” for wrong answers — Activate Learning Guide mode: have it quiz you Socratically on the Renaissance Developer mindsets - EXERCISE 17: Create a Mind Map of how the five AI tools relate to the four competency layers - Pro tips: share notebooks with study partners, export flashcards to Anki, use Debate format for controversial topics	
10:30	Break	15 min
	Refresh	
10:45	Week 1 Retrospective (Whole Cohort)	60 min
	- Sailboat retrospective format: — Wind (propelled us): What worked well? — Anchor (held us back): What was harder than expected? — Rocks (risks ahead): What concerns you about the next 9 weeks? — Island (destination): What are you most excited to learn? - Review cohort norms from Day 1: anything to add or adjust? - Instructor observations: patterns seen across the cohort this week	
11:45	Lunch	60 min
	Form study groups for the weekend	

12:45	Personal Development Map Revisit	60 min
	• EXERCISE 18: Revisit your Day 1 Personal Development Map: • — After a full week: would you change any of your self-ratings? • — What new gaps did you discover? (Often: people realize agent security or CI/CD is weaker than expected) • — Refine your “10-week intention” based on this week’s experience • Peer calibration: share updated maps and compare Day 1 vs Day 5 ratings • Instructor 1:1s begin (5 min each): progress check, concerns, goals	
13:45	Impossible Stuff Day: Self-Directed Exploration	120 min
	• First “Impossible Stuff Day” (recurring every Friday afternoon) • Rules: work on anything that pushes your boundaries. No assigned exercises. • Suggestions: • — Build something in a language you’ve never used, with AI tools only • — Deep-dive Nanobot: add a second skill or connect a second messaging platform • — Explore Antigravity’s Manager View with 2+ parallel agents • — Read the NanoClaw codebase in preparation for Module 4 • — Start your blog: write about the Renaissance Developer mindset shift • Share explorations in the last 15 min — quick round-robin, no slides	
15:45	Break	15 min
	• Stretch	
16:00	Portfolio Polish & Module 2 Preview	30 min
	• EXERCISE 19: Portfolio quality check: • — Clean README with CI badge, setup instructions, project description • — Day 4 app deployed and accessible • — GitHub Projects board with Module 1 issues tracked • — Nanobot fork with custom skill and architecture write-up • Module 2 preview: AI-Augmented Development & the AI Bug Safari • Pre-work: listen to your NotebookLM audio overview, prepare 3 questions for next week	
16:30	Badge Ceremony & Week 1 Close	30 min
	• Badge ceremony: students who completed all deliverables earn the Renaissance Foundations badge • Call out specific achievements: best PR review, most creative ship, best agent skill, best security analysis	

	• Read the Developer Manifesto together: “Polymath — crossing technology and business. Curiosity — never stop exploring. Outcome — accountable for business impact. Precision — communication is productivity. Systems — master complexity.” • The week started with a concept. It ends with proof that you can build. That gap is what this program closes.
--	--

Instructor Notes — Day 5: Day 5 is deliberately lower-intensity. Students need processing time. The Development Map Revisit is the most important assessment activity — it shows whether Day 1’s frameworks actually landed. Students who can articulate specific changes in their self-assessment have internalized the model. The badge ceremony should feel meaningful — read out the Manifesto as a shared ritual. This becomes a recurring closing ceremony each module. The Impossible Stuff Day should feel genuinely free — some students build, some read, some rest. All are valid.

Appendix A: Exercise Summary

#	Day	Exercise	Type
1	1	Personal Development Map: radar chart, competency layers, Strategist Venn, 10-week intention	Reflection
2	1	Cohort Norms: collaborative operating agreement	Collaborative
3	2	First Claude Code session: generate + review code	Hands-on
4	2	Claude Code: generate and improve a GitHub Actions workflow	Hands-on
5	2	Kiro spec-driven build: CSV-to-JSON CLI tool	Hands-on
6	2	Antigravity agent-assisted REST API endpoint	Hands-on
7	2	The Rosetta Stone: same feature, three tools, compare	Analysis
8	2	Start personal prompt library (5 reusable prompts)	Setup
9	3	Fork, read, and map Nanobot architecture	Analysis
10	3	Run Nanobot locally with messaging platform	Setup
11	3	Extend Nanobot with a custom skill	Hands-on
12	3	Security threat model for Nanobot agent	Analysis
13	3	Enhance GitHub Actions pipeline for AI-augmented code	Hands-on
14	4	Architecture planning sprint for ship challenge	Planning
15	4	CI/CD pipeline: lint + test + build + deploy	Hands-on
16	5	Create Renaissance Developer Study Notebook in NotebookLM	Setup
17	5	Mind Map: AI tools × competency layers	Reflection
18	5	Revisit and refine Personal Development Map	Assessment
19	5	Portfolio quality check and polish	Quality

Appendix B: Deliverables Checklist

Complete all items to earn the Renaissance Foundations badge:

1. **Personal Development Map** — Day 1 version + Day 5 revised version, committed to portfolio repo
2. **AI Tool Rosetta Stone** — Same feature built with 3 tools, comparison write-up in repo wiki
3. **Nanobot Agent** — Forked repo with custom skill, architecture write-up, and security threat model
4. **Day 4 Application** — Working app deployed to a public URL with automated CI/CD pipeline
5. **Prompt Library** — 5+ reusable prompts organized by task type in portfolio repo
6. **Blog Post** — “My First AI-Augmented Build” (300–500 words) in repo wiki
7. **NotebookLM Study Notebook** — Loaded with curriculum, Module 2 audio overview generated, flashcards created
8. **GitHub Portfolio** — Clean README with CI badge, Projects board with Module 1 issues tracked

Appendix C: GitHub Skills Add-On Bootcamp

Status: Optional pre-program add-on. Students who are already proficient with Git, GitHub Actions, and GitHub Projects may self-certify and skip. Students who are new to GitHub or need a refresher should complete this before or during Week 1.

Self-Certification Criteria

You may skip this add-on if you can confidently do ALL of the following:

- Create a repository, branch, commit, push, and create a pull request from the command line
- Resolve a merge conflict in a file with competing changes
- Write a GitHub Actions workflow from scratch (YAML) that runs lint and test on push
- Use GitHub Projects to create a board with issues, custom fields, and automation
- Use the GitHub CLI (gh) for basic operations: issue create, pr create, repo clone

Add-On Part 1: Git & GitHub Foundations (Half-Day)

Estimated time: 3–4 hours. Can be completed self-paced or in an instructor-led session.

1. **Git Mental Model (45 min):** Commits as snapshots, branches as pointers, HEAD, the three trees (working directory, staging, repository). Practice: `git log --oneline --graph --all`
2. **Repository Setup (30 min):** Create portfolio repo with README, .gitignore (Node + Python), LICENSE (MIT). Configure .gitconfig with aliases (co, br, st, lg)
3. **GitHub CLI Setup (20 min):** Install gh, authenticate (gh auth login), verify with gh repo view and gh issue list
4. **Branching & PRs (45 min):** GitHub Flow model. Create feature branch, make changes, push, create PR with description template. Practice merge strategies: squash vs. rebase vs. merge commit
5. **The Conflict Gauntlet (30 min):** Fork a provided repo with intentionally conflicting files. Create a branch, make changes, encounter and resolve a merge conflict, submit a PR
6. **Branch Protection (20 min):** Set up rules on main: require PR reviews, require status checks. Create PR template (.github/PULL_REQUEST_TEMPLATE.md) and issue templates

Add-On Part 2: GitHub Actions & Projects (Half-Day)

Estimated time: 3–4 hours. Can be completed self-paced or in an instructor-led session.

1. **Actions Fundamentals (45 min):** Workflow anatomy: triggers (on), jobs, steps, runners, YAML schema. Live-code a complete workflow: checkout → setup-node → install → lint → test. Write from scratch, not copy-paste

2. **Build Your First Pipeline (45 min):** Create `.github/workflows/ci.yml` triggered on push and PR. Add lint, test, build steps. Add artifact upload. Add status badge to README
3. **Advanced Patterns (30 min):** Matrix builds (Node 18/20/22), dependency caching (actions/cache), secrets management. Reusable workflows overview
4. **GitHub Projects Setup (30 min):** Create a project board with columns (Backlog, This Week, In Progress, Review, Done). Add custom fields (Module, Priority, Effort). Set up automation: auto-move on PR create/merge
5. **GitHub MCP Server & Codespaces (20 min):** Demo: using AI agents to manage issues/PRs programmatically. Overview of Codespaces for consistent cloud dev environments
6. **Verification (15 min):** Checklist: green CI pipeline, Projects board with issues, gh CLI working, branch protection enabled. Self-certify or ask an instructor to verify

Instructor Notes for Add-On: This add-on can be offered as: (a) a live half-day session the weekend before the program starts, (b) self-paced pre-work with video walkthroughs, or (c) an evening session during Week 1 for students who need it. The key principle: GitHub skills are necessary but not unique to this program. The core curriculum time should be spent on what IS unique: the Renaissance Developer concept, AI tool mastery, and agent architecture. Students who skip the add-on should still verify their setup with the checklist on Day 1 afternoon.

End of Module 1 Lesson Plan v1.1